

Exam Project Requirements - Programming

Project Overview

Type: Individual or Group (up to 4 students)

Duration: 3+ weeks

Hand-in Date: 21st May 2026

Delivery: ZIP file + report via WISEflow

Exam Format: 30 minutes:

- 5 min presentation
 - 20 min live coding and Q&A
 - 5 min grading
-

Context

The security company Øresund Industrial A/S continues to face security challenges. Building on your work from Mandatory 1 and 2, you must now create a comprehensive threat intelligence analysis system using the live incident API.

Important: The API now generates incidents continuously. If accessed after a month, there may be approximately 2000+ incidents in the system. Your program must handle large datasets efficiently.

Program Requirements

1. API Integration

- Connect to the incident API and retrieve authentication token
- Implement proper token management (storage/renewal)
- Handle large datasets (2000+ incidents) efficiently using pagination
- Use appropriate HTTP request methods with proper headers
- Implement retry logic for failed requests

2. Data Processing

- Parse JSON incident data into Python objects
- Extract and categorize Indicators of Compromise (IOCs)
- Handle missing or malformed data gracefully

3. Database Storage

- Design and implement an SQLite database
- Recommended minimum schema:
 - Incidents table:
 - `incidentId`
 - `incidentName`
 - `severity`
 - `status`
 - `createdTime`
 - Alerts table
 - `alertId`
 - `incidentId`
 - `machineId`
 - `detectionSource`
 - `firstActivity`
 - IOCs (entities) table
 - `incidentId`
 - type (ex: `domains`, `emails`, `fileHashes`, `ips`, `processes`, `urls`, etc.)
 - value
- Handle duplicate data appropriately

4. Error Handling

- Validate all API responses
- Handle network errors and timeouts
- Manage rate limiting (50 requests/minute, 1500 requests/hour)
- Validate data before database insertion
- Implement basic logging (write to a file or use logging module)
 - Log errors, warnings, and important operations

5. Code Quality

- Follow PEP8 conventions
 - Include comprehensive documentation:
 - Function docstrings
 - Inline comments for complex logic
 - Organize code into logical functions
 - Use meaningful variable and function names
 - If using AI for **documentation**, label it clearly
-

Technical Requirements

Required Python Modules

- requests - API communication
- json - JSON parsing
- sqlite3 - Database operations
- datetime - Timestamp handling

Recommended Additional Modules

- logging - Logging
 - time - Rate limiting/delays
-

Deliverables

1. Source Code

- Well-organized Python files
- Configuration file for API settings (without hardcoded tokens)
- Database creation script

2. Written Report (Max 5 pages)

- Problem analysis
- Design decisions and justifications
- Challenges encountered and solutions
- Future improvements

3. Presentation (5 minutes)

Choose to present on:

- System architecture and design
 - Live demonstration of program functionality
 - Technical challenges and solutions
 - Data analysis results and insights
 - Code walkthrough of critical sections
 - Future enhancement proposals
-

Important: When fetching 2000 incidents at 100 per request:

- Requires 20 API calls
 - Takes approximately 24-30 seconds (with rate limiting)
 - Plan your implementation accordingly
 - And remember we don't want duplicates in the database
-

Security Considerations

- Never hardcode API tokens in source code
 - Use environment variables or configuration files (e.g. .env files - this was not covered during the course, research it yourself)
 - Implement proper error messages without exposing sensitive info
 - Handle API credentials securely
-

Resources

- **API Documentation:** Student_API_Guide (on ITSLearning)
 - **Python Documentation:** <https://docs.python.org/3/>
 - **SQLite Documentation:** <https://docs.python.org/3/library/sqlite3.html>
 - **Requests Library:** <https://requests.readthedocs.io/>
 - **PEP8 Style Guide:** <https://pep8.org/>
-

Good luck with your exam project!